

服务端实习面试 - MySQL 高频50题

题库总览

模块	题号	主题	难度
模块一	1-10	SQL基础查询	入门
模块二	11-20	多表查询与子查询	进阶
模块三	21-30	索引与查询优化	核心
模块四	31-40	事务与锁机制	深入
模块五	41-50	数据库设计与进阶	综合

每道题包含： 题干描述 + 参考SQL/答案 + 逻辑解析 + 易错点分析

面试高频 Top10（优先复习）

- 索引类型与B+树原理（题21、27）
- 事务隔离级别（题32）
- MVCC机制（题33）
- 索引失效场景（题25）
- 最左前缀原则（题23）
- EXPLAIN使用（题24）
- 慢SQL优化（题28）
- 锁的类型（题34）
- 主从复制原理（题45）
- 分页优化（题48）

模块一：SQL基础查询（第1-10题）

题目1：查询第二高的薪水

题干：编写SQL查询，获取 `Employee` 表中第二高的薪水。如果不存在第二高的薪水，查询应返回 `NULL`。

[复制代码](#)

Employee 表:

id	salary
1	100
2	200
3	300

参考SQL：

[复制代码](#)

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

另一种写法（使用 LIMIT + OFFSET）：

[复制代码](#)

```
SELECT
  (SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1
  OFFSET 1)
AS SecondHighestSalary;
```

逻辑解析：

- 方法一：先子查询找到最高薪水，再在排除最高薪水的情况下取最大值
- 方法二：对薪水去重排序后取第二条记录
- 使用子查询包裹可以保证无结果时返回 `NULL` 而非空结果集

易错点：

- 忘记处理不存在第二高薪水时应返回 `NULL` 的情况
- 未使用 `DISTINCT` 导致重复薪水干扰排名
- 直接用 `LIMIT 1 OFFSET 1` 不包裹子查询会返回空行而非 `NULL`

题目2：查找重复的电子邮箱

题干：编写SQL查询，查找 `Person` 表中所有重复的电子邮箱。

[复制代码](#)

Person 表:

id	email
1	a@b.com
2	c@d.com
3	a@b.com

参考SQL：

[复制代码](#)

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

逻辑解析：

- GROUP BY 按 email 分组，HAVING 过滤出出现次数大于1的组
- HAVING 与 WHERE 的区别：WHERE 在分组前过滤，HAVING 在分组后过滤

易错点：

- 混淆 WHERE 和 HAVING 的使用时机
- 使用 COUNT(email) 替代 COUNT(*)，在 email 为 NULL 时可能产生差异
- 忘记 GROUP BY 直接使用聚合函数

题目3：从不订购的客户

题干：找出从不订购任何东西的客户。

[复制代码](#)

Customers 表:

id	name
----	------

id	name
1	Joe
2	Henry
3	Sam
4	Max

Orders 表:

id	customerId
1	3
2	1

参考SQL:


[复制代码](#)

```
SELECT name AS Customers
FROM Customers
WHERE id NOT IN (SELECT customerId FROM Orders);
```

另一种写法 (LEFT JOIN) :


[复制代码](#)

```
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customerId
WHERE o.id IS NULL;
```

逻辑解析:

- NOT IN: 子查询取出所有下过单的客户ID, 主查询排除这些ID
- LEFT JOIN: 左连接后筛选右表为 NULL 的记录, 表示没有匹配到订单

易错点:

- NOT IN 子查询中如果存在 NULL 值会导致整个结果为空 (MySQL中 NOT IN 对 NULL 敏感)
- LEFT JOIN 时忘记判断右表字段 IS NULL 而非左表字段
- 使用 NOT EXISTS 写法时忘记关联条件

题目4：组合两个表

题干：编写SQL查询，报告 `Person` 表中每个人的姓、名、城市和州。如果 `personId` 的地址不在 `Address` 表中，则报告为 `NULL`。

[复制代码](#)

Person 表:

personId	lastName	firstName
1	Wang	Allen
2	Alice	Bob

Address 表:

addressId	personId	city	state
1	2	New York City	NY
2	3	Leetcode	CA

参考SQL:

[复制代码](#)

```
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON p.personId = a.personId;
```

逻辑解析:

- 使用 `LEFT JOIN` 确保 `Person` 表中所有记录都会出现在结果中
- 如果某人没有地址记录，`city` 和 `state` 自动为 `NULL`
- 不能用 `INNER JOIN`，否则无地址的人会被过滤掉

易错点:

- 使用 `INNER JOIN` 导致无地址的人缺失
- `JOIN` 条件写错（应关联 `personId`）
- 忘记 `Person` 应为主表（`LEFT JOIN` 左侧）

题目5：删除重复的电子邮箱

题干：编写SQL删除 `Person` 表中所有重复的电子邮箱，只保留每个重复邮箱中 id 最小的那一条。

[复制代码](#)

Person 表：

id	email
1	john@example.com
2	bob@example.com
3	john@example.com

参考SQL：

[复制代码](#)

```
DELETE p1
FROM Person p1, Person p2
WHERE p1.email = p2.email AND p1.id > p2.id;
```

另一种写法：

[复制代码](#)

```
DELETE FROM Person
WHERE id NOT IN (
    SELECT * FROM (
        SELECT MIN(id) FROM Person GROUP BY email
    ) AS tmp
);
```

逻辑解析：

- 方法一：自连接，保留相同 email 中 id 更小的记录，删除 id 大的
- 方法二：子查询找出每组 email 中最小的 id，删除不在该集合中的记录
- MySQL 不允许在 DELETE 的 WHERE 子句中直接引用同一个表的子查询，所以需要嵌套一层临时表

易错点：

- MySQL 不能在 DELETE 的子查询中直接 SELECT 同一张表（需要额外嵌套一层）

- 自连接时 DELETE 后面要明确指定删除哪个别名 (p1 而非 p2)
- 忘记条件 p1.id > p2.id 导致删除了所有记录

题目6：超过经理收入的员工

题干：编写SQL查询，找出收入超过其经理的员工。

[复制代码](#)

Employee 表:

id	name	salary	managerId
1	Joe	70000	3
2	Henry	80000	4
3	Sam	60000	NULL
4	Max	90000	NULL

参考SQL:

[复制代码](#)

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

逻辑解析:

- 自连接：将 Employee 表与自身连接，一方代表员工，一方代表经理
- JOIN 条件：员工的 managerId 等于经理的 id
- WHERE 条件：员工薪水大于经理薪水

易错点:

- 使用 LEFT JOIN 会引入 managerId 为 NULL 的记录（无经理的员工），需额外过滤
- JOIN 条件方向写反：应该是 e.managerId = m.id，而非 e.id = m.managerId
- 忘记表别名导致字段歧义

题目7：部门工资最高的员工

题干：编写SQL查询，找出每个部门中薪资最高的员工。



复制代码

Employee 表:

id	name	salary	departmentId
1	Joe	70000	1
2	Jim	90000	1
3	Henry	80000	2
4	Sam	60000	2
5	Max	90000	1

Department 表:

id	name
1	IT
2	Sales

参考SQL:



复制代码

```
SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary
FROM Employee e
JOIN Department d ON e.departmentId = d.id
WHERE (e.departmentId, e.salary) IN (
    SELECT departmentId, MAX(salary)
    FROM Employee
    GROUP BY departmentId
);
```

逻辑解析:

- 子查询先按部门分组找出每个部门的最高薪资
- 外层查询用 (departmentId, salary) 的元组匹配，找出所有拿到最高薪资的员工
- 如果一个部门有多人并列最高，都会被查出来

易错点:

- 只取一条记录（使用 LIMIT）导致并列最高薪水的员工被遗漏

- 忘记关联 Department 表获取部门名称
- 使用 GROUP BY 后直接 SELECT 非聚合列（非 MySQL 严格模式下可能不报错但结果不正确）

题目8：第N高的薪水（通用化）

题干：编写一个SQL函数/查询，获取 Employee 表中第 N 高的薪水。如果不存在，返回 NULL。

参考SQL：



复制代码

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
  SET N = N - 1;
  RETURN (
    SELECT DISTINCT salary FROM Employee
    ORDER BY salary DESC
    LIMIT 1 OFFSET N
  );
END
```

如果不用函数：



复制代码

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET N-1; -- N为具体数字时替换
```

逻辑解析：

- OFFSET 是从0开始的，所以第N高需要 OFFSET N-1
- DISTINCT 去重保证排名不受重复值影响
- 使用函数封装时需要在函数体内做 $N = N - 1$ 的处理

易错点：

- LIMIT/OFFSET 不支持表达式（如 N-1），需要先用变量计算
- 忘记 DISTINCT 导致重复薪水占排名
- 不存在第N高时没有返回 NULL 的处理

题目9：分数排名

题干：编写SQL查询，对分数进行排名。排名按分数从高到低排列，相同分数排名相同，且排名连续（即 dense_rank）。

[复制代码](#)

Scores 表：

id	score
1	3.50
2	3.65
3	4.00
4	3.85
5	4.00
6	3.65

参考SQL：

[复制代码](#)

```
SELECT score,
       DENSE_RANK() OVER (ORDER BY score DESC) AS 'rank'
FROM Scores;
```

不使用窗口函数的写法：

[复制代码](#)

```
SELECT s.score,
       (SELECT COUNT(DISTINCT s2.score) FROM Scores s2 WHERE s2.score
        >= s.score) AS 'rank'
FROM Scores s
ORDER BY s.score DESC;
```

逻辑解析：

- DENSE_RANK(): 密集排名，相同值排名相同，后续排名不跳跃
- 区别于 RANK()（跳跃排名）和 ROW_NUMBER()（唯一排名）
- 子查询方法：统计大于等于当前分数的不同分数数量即为排名

易错点：

- 混淆 DENSE_RANK、RANK、ROW_NUMBER 三者区别
- 使用 RANK() 导致排名不连续（如 1,1,3 而非 1,1,2）
- rank 是 MySQL 关键字，需要加引号或反引号

题目10：连续出现的数字

题干：找出至少连续出现三次的数字。



复制代码

Logs 表：

id	num
1	1
2	1
3	1
4	2
5	1
6	2
7	2

参考SQL：



复制代码

```
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM Logs l1, Logs l2, Logs l3
WHERE l1.id = l2.id - 1
      AND l2.id = l3.id - 1
      AND l1.num = l2.num
      AND l2.num = l3.num;
```

使用窗口函数：



复制代码

```
SELECT DISTINCT num AS ConsecutiveNums
FROM (
  SELECT num,
         LAG(num, 1) OVER (ORDER BY id) AS prev1,
```

```
LAG(num, 2) OVER (ORDER BY id) AS prev2
FROM Logs
) t
WHERE num = prev1 AND num = prev2;
```

逻辑解析：

- 方法一：三表自连接，确保连续三个 id 的 num 相同
- 方法二：窗口函数 LAG 获取前两行的值，比较三者是否相等
- DISTINCT 去重避免连续4次以上出现时重复输出

易错点：

- 假设 id 一定连续（实际可能有间断），自连接方法在 id 不连续时失效
- 忘记 DISTINCT 导致输出重复
- LAG 函数的 OVER 子句中忘记 ORDER BY

模块二：多表查询与子查询（第11-20题）

题目11：部门工资前三高的员工

题干：编写SQL查询，找出每个部门中工资前三高的员工（如有并列，全部列出）。

参考SQL：

∨ 复制代码

```
SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary
FROM Employee e
JOIN Department d ON e.departmentId = d.id
WHERE (
    SELECT COUNT(DISTINCT e2.salary)
    FROM Employee e2
    WHERE e2.departmentId = e.departmentId AND e2.salary > e.salary
) < 3;
```

使用窗口函数：

∨ 复制代码

```
SELECT Department, Employee, Salary
FROM (
    SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary,
```

```

        DENSE_RANK() OVER (PARTITION BY e.departmentId ORDER BY
e.salary DESC) AS rk
    FROM Employee e
    JOIN Department d ON e.departmentId = d.id
) ranked
WHERE rk <= 3;

```

逻辑解析：

- 子查询方法：对每个员工，统计同部门中比他工资高的不同工资数量，小于3说明在前三
- 窗口函数方法：按部门分区，薪水降序做 DENSE_RANK，取排名 <= 3 的记录
- DENSE_RANK 保证并列的都能入选

易错点：

- 使用 ROW_NUMBER 代替 DENSE_RANK 会漏掉并列的员工
- PARTITION BY 忘记按部门分区，变成全局排名
- 相关子查询性能较差，大数据量建议用窗口函数

题目12：换座位

题干：编写SQL查询来交换每两个连续学生的座位号。如果学生人数是奇数，则最后一个学生的座位号不变。



复制代码

Seat 表：

```

+----+-----+
| id | student |
+----+-----+
| 1  | Abbot   |
| 2  | Doris   |
| 3  | Emerson |
| 4  | Green   |
| 5  | Jeames  |
+----+-----+

```

参考SQL：



复制代码

```

SELECT
CASE
    WHEN id % 2 = 1 AND id = (SELECT MAX(id) FROM Seat) THEN id

```

```
    WHEN id % 2 = 1 THEN id + 1
    ELSE id - 1
END AS id,
student
FROM Seat
ORDER BY id;
```

逻辑解析：

- 奇数 id：与下一个交换 (id+1) ，除非是最后一个
- 偶数 id：与上一个交换 (id-1)
- 最后一个且为奇数：保持不变
- 最终按新 id 排序输出

易错点：

- 忘记处理奇数个学生时最后一位不需要交换的情况
- CASE WHEN 顺序写错导致最后一位奇数先命中了 id+1 的条件
- 结果没有 ORDER BY 导致顺序混乱

题目13：行程和用户（取消率计算）

题干：编写SQL查询，计算 2013-10-01 至 2013-10-03 期间，非禁止用户的行程取消率（保留两位小数）。

[复制代码](#)

Trips 表包含: id, client_id, driver_id, city_id, status, request_at
Users 表包含: users_id, banned, role

取消率 = 取消的行程数 / 总行程数

参考SQL：

[复制代码](#)

```
SELECT t.request_at AS Day,
       ROUND(
           SUM(CASE WHEN t.status != 'completed' THEN 1 ELSE 0 END) /
COUNT(*),
           2
       ) AS 'Cancellation Rate'
FROM Trips t
JOIN Users u1 ON t.client_id = u1.users_id AND u1.banned = 'No'
```

```
JOIN Users u2 ON t.driver_id = u2.users_id AND u2.banned = 'No'
WHERE t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY t.request_at;
```

逻辑解析：

- 双重 JOIN Users 表：既要排除被禁止的客户，也要排除被禁止的司机
- CASE WHEN 统计非 completed 状态的行程作为取消数
- ROUND 保留两位小数
- GROUP BY 按日期分组计算每天的取消率

易错点：

- 只排除了被禁止的客户，忘记也需要排除被禁止的司机
- 日期筛选条件放错位置（应放 WHERE 而非 HAVING）
- 除法结果为0时的精度问题

题目14：游戏玩法分析（首次登录）

题干：编写SQL查询，报告每个玩家首次登录平台的日期。



复制代码

Activity 表：

player_id	device_id	event_date	games_played
1	2	2016-03-01	5
1	2	2016-05-02	6
2	3	2017-06-25	1
3	1	2016-03-02	0
3	4	2018-07-03	5

参考SQL：



复制代码

```
SELECT player_id, MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

逻辑解析：

- 按 player_id 分组，取每组最早的日期即为首次登录
- MIN() 函数对日期类型直接适用

易错点：

- 使用 ORDER BY + LIMIT 只能取全局首条，不是每个玩家的首次
- 尝试 SELECT 非分组列（如 device_id）在严格模式下会报错
- 日期比较时格式不一致可能导致错误

题目15：树节点类型判断

题干：给定一个表 Tree，每个节点有 id 和 p_id（父节点），判断每个节点是 Root、Inner 还是 Leaf。



复制代码

Tree 表：

```
+-----+-----+
| id | p_id |
+-----+-----+
| 1  | NULL |
| 2  | 1    |
| 3  | 1    |
| 4  | 2    |
| 5  | 2    |
+-----+-----+
```

参考SQL：



复制代码

```
SELECT id,
CASE
    WHEN p_id IS NULL THEN 'Root'
    WHEN id IN (SELECT DISTINCT p_id FROM Tree WHERE p_id IS NOT NULL)
THEN 'Inner'
    ELSE 'Leaf'
END AS type
FROM Tree;
```

逻辑解析：

- p_id 为 NULL 的是根节点

- 作为其他节点的父节点（出现在 p_id 列中）且自身有父节点的是内部节点
- 既有父节点又不是任何节点的父节点的是叶子节点

易错点：

- 子查询中忘记排除 p_id IS NOT NULL 可能导致误判
- CASE WHEN 的判断顺序很重要，Root 必须最先判断
- 只有一个节点时它既是 Root 也满足 Leaf 的条件，需要正确排列优先级

题目16：查询近30天活跃用户数

题干：编写SQL查询，计算截至 2019-07-27（含）的近30天内每天的活跃用户数。



复制代码

Activity 表包含: user_id, session_id, activity_date, activity_type

参考SQL：



复制代码

```
SELECT activity_date AS day, COUNT(DISTINCT user_id) AS active_users
FROM Activity
WHERE activity_date BETWEEN DATE_SUB('2019-07-27', INTERVAL 29 DAY) AND
'2019-07-27'
GROUP BY activity_date;
```

逻辑解析：

- BETWEEN 包含两端，近30天应为当天减29天到当天
- COUNT(DISTINCT user_id) 去重计算每天的独立用户数
- DATE_SUB 函数计算日期偏移

易错点：

- "近30天"是30天的区间（减29天），而非减30天
- 忘记 DISTINCT 导致同一用户多次活动被重复计数
- 日期格式错误或时区问题

题目17：文章浏览（自己看自己的文章）

题干：找出所有浏览过自己文章的作者，结果按 id 升序。

[复制代码](#)

Views 表:

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
3	4	4	2019-07-21

参考SQL:

[复制代码](#)

```
SELECT DISTINCT author_id AS id
FROM Views
WHERE author_id = viewer_id
ORDER BY id;
```

逻辑解析:

- 筛选 `author_id = viewer_id` 即为自己浏览自己的文章
- `DISTINCT` 去重 (同一作者可能多次浏览自己文章)
- `ORDER BY` 按要求升序排列

易错点:

- 忘记 `DISTINCT` 导致结果重复
- 排序字段写成 `author_id` 而别名为 `id`, 部分场景可能出错
- 把条件写成 `article_id = viewer_id` (逻辑错误)

题目18: 每台机器的进程平均运行时间

题干: 编写SQL查询, 计算每台机器上每个进程的平均运行时间 (结束时间 - 开始时间), 保留3位小数。

[复制代码](#)

Activity 表: machine_id, process_id, activity_type('start'/'end'), timestamp

参考SQL:

[复制代码](#)

```
SELECT machine_id,
       ROUND(AVG(end_time - start_time), 3) AS processing_time
FROM (
  SELECT machine_id, process_id,
         MAX(CASE WHEN activity_type = 'start' THEN timestamp END) AS
start_time,
         MAX(CASE WHEN activity_type = 'end' THEN timestamp END) AS
end_time
  FROM Activity
  GROUP BY machine_id, process_id
) t
GROUP BY machine_id;
```

另一种写法（自连接）：

[复制代码](#)

```
SELECT a1.machine_id,
       ROUND(AVG(a2.timestamp - a1.timestamp), 3) AS processing_time
FROM Activity a1
JOIN Activity a2
  ON a1.machine_id = a2.machine_id
  AND a1.process_id = a2.process_id
  AND a1.activity_type = 'start'
  AND a2.activity_type = 'end'
GROUP BY a1.machine_id;
```

逻辑解析：

- 方法一：行转列，将 start 和 end 时间转到同一行再计算差值
- 方法二：自连接匹配同一机器同一进程的 start 和 end 记录
- 最外层按 machine_id 分组求平均

易错点：

- 自连接时忘记限定 process_id 相同

- ROUND 的精度参数写错
- 没有正确处理一台机器多个进程的情况

题目19：按日期分组销售产品

题干：编写SQL查询，按日期分组，返回每天销售的不同产品数量及其名称（按字典序逗号分隔）。

[复制代码](#)

Activities 表:

sell_date	product
2020-05-30	Headphone
2020-06-01	Pencil
2020-06-02	Mask
2020-05-30	Basketball
2020-06-01	Bible
2020-06-02	Mask
2020-05-30	T-Shirt

参考SQL：

[复制代码](#)

```
SELECT sell_date,
       COUNT(DISTINCT product) AS num_sold,
       GROUP_CONCAT(DISTINCT product ORDER BY product SEPARATOR ',') AS
products
FROM Activities
GROUP BY sell_date
ORDER BY sell_date;
```

逻辑解析：

- GROUP_CONCAT 将分组内的多个值连接成一个字符串
- DISTINCT 去重，ORDER BY 保证字典序
- SEPARATOR 指定分隔符（默认为逗号）

易错点：

- 忘记在 GROUP_CONCAT 内部使用 DISTINCT 和 ORDER BY

- GROUP_CONCAT 默认最大长度1024，数据量大时会被截断
- 输出的产品名没有按字典序排列

题目20：每位学生的最高成绩所在科目

题干：编写SQL查询，报告每个学生获得最高成绩的科目名称。如果有并列，选科目名字典序最小的。

[复制代码](#)

Enrollments 表: student_id, course_id, grade

参考SQL：

[复制代码](#)

```
SELECT student_id, MIN(course_id) AS course_id, grade
FROM Enrollments
WHERE (student_id, grade) IN (
    SELECT student_id, MAX(grade)
    FROM Enrollments
    GROUP BY student_id
)
GROUP BY student_id, grade
ORDER BY student_id;
```

使用窗口函数：

[复制代码](#)

```
SELECT student_id, course_id, grade
FROM (
    SELECT *,
        ROW_NUMBER() OVER (
            PARTITION BY student_id
            ORDER BY grade DESC, course_id ASC
        ) AS rn
    FROM Enrollments
) ranked
WHERE rn = 1;
```

逻辑解析：

- 先找每个学生的最高分，再在最高分并列中取 course_id 最小的
- 窗口函数方法：按学生分区，先按成绩降序再按 course_id 升序，取第一条
- ROW_NUMBER 保证每个学生只取一条

易错点：

- 并列最高分时忘记选字典序最小的科目
- 使用 MAX(grade) 后直接 SELECT course_id 在严格模式下会报错
- 窗口函数排序条件不完整导致结果不确定

模块三：索引与查询优化（第21-30题）

题目21：什么是索引？MySQL 中有哪些索引类型？

题干：请解释 MySQL 索引的概念，并说明常见的索引类型及其适用场景。

参考答案：

⌵

复制代码

```
-- 创建普通索引
CREATE INDEX idx_name ON users(name);

-- 创建唯一索引
CREATE UNIQUE INDEX idx_email ON users(email);

-- 创建复合索引
CREATE INDEX idx_name_age ON users(name, age);

-- 创建前缀索引
CREATE INDEX idx_address ON users(address(10));

-- 查看表索引
SHOW INDEX FROM users;
```

逻辑解析：

索引是帮助 MySQL 高效获取数据的数据结构（类比书的目录）。

索引类型	说明	适用场景
B+Tree索引	默认索引类型，有序	范围查询、排序、精确匹配

Hash索引	基于哈希表	等值查询（Memory引擎）
全文索引	文本搜索	LIKE '%keyword%' 模糊搜索
唯一索引	值不允许重复	邮箱、手机号等唯一字段
复合索引	多列组合	多条件查询
前缀索引	只索引字段前N个字符	长文本字段

易错点：

- 混淆"索引类型"（B+Tree/Hash）与"索引分类"（主键/唯一/普通/全文）
- 认为 Hash 索引支持范围查询（实际不支持）
- 忽视索引的维护成本（增删改会更新索引）

题目22：聚簇索引和非聚簇索引的区别？

题干：请解释聚簇索引（Clustered Index）和非聚簇索引（Secondary Index）的区别。

参考答案：

✓ 复制代码

```
-- InnoDB主键即为聚簇索引
CREATE TABLE users (
  id INT PRIMARY KEY,          -- 聚簇索引
  name VARCHAR(50),
  INDEX idx_name(name)        -- 非聚簇索引（二级索引）
);

-- 通过二级索引查询流程：
-- 1. 在 idx_name 的B+树中找到 name='Alice' 对应的主键 id=5
-- 2. 回表：用主键 id=5 在聚簇索引中找到完整行数据
```

逻辑解析：

特性	聚簇索引	非聚簇索引
数据存储	叶子节点存储完整行数据	叶子节点存储主键值
数量	每表只能有一个	可以有多个

查询4	否	缺少最左列a
查询5	部分 (a)	a可用，跳过b后c无法用
查询6	是 (a,b)	优化器会调整顺序

逻辑解析：

- 最左前缀原则：复合索引从最左列开始匹配，不能跳过中间列
- 查询条件中列的书写顺序不影响（优化器会重排），但必须包含最左列
- 范围查询（>、<、BETWEEN、LIKE 'x%'）之后的列无法使用索引

易错点：

- 认为 WHERE 子句中列的书写顺序必须和索引定义顺序一致
- 忽视范围查询会中断后续列的索引使用
- 误以为 WHERE b=2 也能走 INDEX(a,b,c)（缺少最左列不行）

题目24：EXPLAIN 怎么看？各字段含义是什么？

题干：执行 `EXPLAIN SELECT * FROM users WHERE name = 'Alice'`，解释返回结果中各字段的含义。

参考答案：

∨ 复制代码

```
EXPLAIN SELECT * FROM users WHERE name = 'Alice';
```

字段	含义	关注点
id	查询序号	相同id从上往下执行，不同id大的先执行
select_type	查询类型	SIMPLE/PRIMARY/SUBQUERY/DERIVED
table	查询的表	
type	访问类型	最重要 ，从好到差： system > const > eq_ref > ref > range > index > ALL

possible_keys	可能用到的索引	
key	实际使用的索引	NULL 表示全表扫描
key_len	使用索引的长度	越短越好（联合索引部分使用）
rows	预估扫描行数	越少越好
filtered	按条件过滤后的行百分比	越大越好
Extra	额外信息	Using index(覆盖索引)、 Using filesort(需排序)、 Using temporary(临时表)

逻辑解析：

- **type** 字段最关键：ALL 意味着全表扫描，应尽量优化到 ref 或 range 以上
- **Extra** 中的 Using filesort 和 Using temporary 通常意味着需要优化
- **key** 为 NULL 说明没走索引，需检查索引是否存在或条件是否命中

易错点：

- 只关注是否用了索引，忽视 type 的级别
- 不理解 rows 是估算值而非精确值
- 看到 Using index 以为没走索引（实际是覆盖索引，性能反而好）
- 混淆 possible_keys 和 key（前者是候选，后者是实际使用的）

题目25：索引失效的常见场景

题干：列举MySQL中导致索引失效的常见场景，并给出示例。

参考答案：

⌵

📄 复制代码

```
-- 假设 users 表有索引 INDEX(name), INDEX(age), INDEX(phone)

-- 1. 对索引列使用函数或运算
SELECT * FROM users WHERE YEAR(create_time) = 2024;      -- 失效
SELECT * FROM users WHERE create_time >= '2024-01-01'
    AND create_time < '2025-01-01';                      -- 有效

-- 2. 隐式类型转换
```

```

SELECT * FROM users WHERE phone = 13800138000;           -- 失效（phone是
VARCHAR）
SELECT * FROM users WHERE phone = '13800138000';         -- 有效

-- 3. LIKE 以通配符开头
SELECT * FROM users WHERE name LIKE '%张';              -- 失效
SELECT * FROM users WHERE name LIKE '张%';              -- 有效

-- 4. OR 连接非索引列
SELECT * FROM users WHERE name = '张三' OR address = '北京'; -- 如address
无索引则全表扫描

-- 5. NOT IN / NOT EXISTS（部分场景）
SELECT * FROM users WHERE id NOT IN (1, 2, 3);           -- 主键可能走索引
SELECT * FROM users WHERE name NOT IN ('a', 'b');         -- 可能失效

-- 6. IS NULL / IS NOT NULL（视数据分布）
SELECT * FROM users WHERE name IS NOT NULL;              -- 大量非NULL时可
能全表扫描

```

逻辑解析：

- 索引失效的本质：条件无法利用B+树的有序性进行快速定位
- 函数/运算改变了索引列的值，破坏了有序性
- 隐式转换导致MySQL对索引列做了类型转换函数
- 前缀通配符无法利用索引的前缀匹配特性

易错点：

- 认为加了索引就一定会走索引（优化器可能判断全表扫描更快）
- 不注意隐式类型转换（数字和字符串比较时）
- 对 NULL 值的索引行为不清楚（B+树可以索引 NULL）
- 混淆"索引失效"和"优化器选择不用索引"

题目26：覆盖索引是什么？如何利用覆盖索引优化查询？

题干：解释覆盖索引的概念，并举例说明如何通过覆盖索引避免回表。

参考答案：

复制代码

```

-- 表结构
CREATE TABLE orders (
  id INT PRIMARY KEY,

```

```

user_id INT,
amount DECIMAL(10,2),
status TINYINT,
INDEX idx_user_status(user_id, status)
);

-- 需要回表的查询 (SELECT * 包含 amount 不在索引中)
SELECT * FROM orders WHERE user_id = 100 AND status = 1;

-- 覆盖索引查询 (查询列全在索引中, 无需回表)
SELECT user_id, status FROM orders WHERE user_id = 100 AND status = 1;
-- EXPLAIN 的 Extra 会显示 "Using index"

-- 优化: 如果经常需要 amount, 可以建联合索引
CREATE INDEX idx_user_status_amount ON orders(user_id, status, amount);
-- 现在即使 SELECT amount 也能覆盖索引
SELECT amount FROM orders WHERE user_id = 100 AND status = 1;

```

逻辑解析:

- 覆盖索引: 查询所需的所有列都包含在索引中, 无需回表到聚簇索引
- EXPLAIN 的 Extra 列显示 "Using index" 即表示使用了覆盖索引
- 好处: 减少IO次数, 避免随机读取聚簇索引页

易错点:

- 将覆盖索引和复合索引混为一谈 (覆盖索引是一种使用方式, 不是索引类型)
- 为了覆盖索引盲目加列到索引中 (会增大索引体积、影响写入性能)
- 不清楚 SELECT * 基本无法利用覆盖索引

题目27: 为什么MySQL用B+树而不用B树或Hash?

题干: 请对比B+树、B树和Hash结构, 说明MySQL InnoDB为什么选择B+树作为索引结构。

参考答案:

特性	B+树	B树	Hash
范围查询	高效 (叶子链表)	需要中序遍历	不支持
等值查询	$O(\log N)$	$O(\log N)$	$O(1)$
排序	天然有序	需要额外排序	不支持

磁盘IO	低（非叶子不存数据）	高（每个节点存数据）	低
稳定性	所有查询都到叶子节点	深度不固定	有hash冲突

逻辑解析：

- B+树非叶子节点只存键值不存数据，单个页能存更多索引项 -> 树更矮 -> 磁盘IO更少
- B+树叶节点通过双向链表连接 -> 范围查询只需遍历链表，效率极高
- B+树所有数据都在叶子节点 -> 查询性能稳定（每次都是从根到叶子）
- Hash 只适合等值查询，不支持范围查询和排序

易错点：

- 认为Hash索引查询一定比B+树快（Hash有冲突，且不支持范围/排序）
- 不清楚B+树和B树的核心区别（非叶子是否存数据、叶子是否有链表）
- 忽视磁盘IO是数据库性能的关键瓶颈

题目28：如何优化一个慢SQL查询？

题干：给定一个慢查询，描述你的分析和优化步骤。

[复制代码](#)

```
-- 慢查询示例
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.create_time > '2024-01-01'
      AND u.status = 1
GROUP BY u.id
ORDER BY order_count DESC
LIMIT 10;
```

参考答案：

[复制代码](#)

```
-- 步骤1: EXPLAIN 分析
EXPLAIN SELECT ...;

-- 步骤2: 检查索引
```

```
-- orders 表应有 (user_id, create_time) 复合索引
CREATE INDEX idx_orders_user_time ON orders(user_id, create_time);
-- users 表应有 status 索引
CREATE INDEX idx_users_status ON users(status);

-- 步骤3: 优化SQL
-- 问题: LEFT JOIN + WHERE条件在右表上 = 隐式INNER JOIN
-- 修正为明确的 INNER JOIN 或将条件移到 ON 中
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
INNER JOIN orders o ON u.id = o.user_id AND o.create_time > '2024-01-01'
WHERE u.status = 1
GROUP BY u.id
ORDER BY order_count DESC
LIMIT 10;

-- 步骤4: 考虑是否需要覆盖索引
CREATE INDEX idx_orders_cover ON orders(user_id, create_time, id);
```

逻辑解析：

优化步骤（通用方法论）：

1. **EXPLAIN** 看执行计划：关注 type、rows、Extra
2. **检查索引**：WHERE/JOIN/ORDER BY/GROUP BY 涉及的字段是否有索引
3. **分析SQL语义**：是否有逻辑可简化、是否存在隐式转换
4. **调整SQL写法**：避免 SELECT *、减少子查询、优化 JOIN 顺序
5. **数据层面**：考虑分页、分表、缓存等架构层优化

易错点：

- LEFT JOIN 后在 WHERE 中过滤右表条件 = 隐式 INNER JOIN（语义错误）
- 只加索引不分析SQL本身的逻辑问题
- GROUP BY 和 ORDER BY 使用不同字段时会产生 filesort
- 忽略 LIMIT 对优化器的提示作用

题目29：什么是索引下推（Index Condition Pushdown）？

题干：请解释 MySQL 5.6 引入的索引下推（ICP）优化机制。

参考答案：



复制代码

```
-- 假设有复合索引 INDEX(name, age)
```

```
-- 查询
SELECT * FROM users WHERE name LIKE '张%' AND age = 25;

-- 无ICP的执行流程：
-- 1. 索引中用 name LIKE '张%' 找到所有姓张的记录的主键
-- 2. 逐条回表读取完整行
-- 3. 在Server层用 age=25 过滤

-- 有ICP的执行流程：
-- 1. 索引中用 name LIKE '张%' 定位记录
-- 2. 在索引层直接用 age=25 过滤（因为age也在索引中）
-- 3. 只对满足条件的记录回表
-- 好处：减少回表次数

-- EXPLAIN中 Extra 显示 "Using index condition" 表示使用了ICP
```

逻辑解析：

- 传统流程：存储引擎只负责按索引取数据，过滤条件由Server层处理
- ICP优化：将部分 WHERE 条件下推到存储引擎层，在索引中就完成过滤
- 适用条件：WHERE 中有索引列的条件，但该条件不能用于索引查找（如复合索引中范围条件后的列）

易错点：

- 混淆"索引下推"和"覆盖索引"（ICP仍需要回表，只是减少了回表次数）
- 不清楚ICP在什么场景下触发（复合索引 + 范围条件后的列判断）
- 认为所有条件都能下推（只有索引中包含的列才能下推）

题目30：count(*)、count(1)、count(列名) 的区别

题干：解释 COUNT(*)、COUNT(1) 和 COUNT(列名) 的区别和性能差异。

参考答案：

复制代码

```
-- 假设表中有NULL值
SELECT COUNT(*) FROM users; -- 统计总行数（包含NULL行）
SELECT COUNT(1) FROM users; -- 统计总行数（包含NULL行）
SELECT COUNT(name) FROM users; -- 统计name列非NULL的行数

-- 性能对比（InnoDB）：
-- COUNT(*) ≈ COUNT(1) > COUNT(主键) > COUNT(普通列)
-- MySQL对COUNT(*)做了专门优化，会选择最小的非聚簇索引来计数
```

逻辑解析：

写法	含义	是否统计NULL
COUNT(*)	统计行数	是
COUNT(1)	统计行数	是
COUNT(列名)	统计该列非NULL的值数量	否

性能说明：

- InnoDB 的 COUNT(*) 不像 MyISAM 有直接维护行数的变量，需要逐行扫描
- MySQL优化器会为 COUNT(*) 选择最小的索引树来遍历（非聚簇索引树小）
- COUNT(*) 和 COUNT(1) 在MySQL中已被优化为等价，无性能差异

易错点：

- 认为 COUNT(1) 比 COUNT(*) 快（在MySQL中无差异）
- 不清楚 COUNT(列名) 会忽略 NULL 值
- 以为 InnoDB 的 COUNT(*) 像 MyISAM 一样是 O(1)
- 大表 COUNT(*) 性能差时不知道用近似值（如 SHOW TABLE STATUS）

模块四：事务与锁机制（第31-40题）

题目31：什么是事务？事务的四大特性（ACID）是什么？

题干：解释数据库事务的概念及其四大特性。

参考答案：



复制代码

```
-- 事务示例：转账操作
START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE user_id = 2;

-- 如果两条都成功
COMMIT;
-- 如果有任何失败
```

```
-- ROLLBACK;
```

逻辑解析：

ACID 四大特性：

特性	英文	含义	实现机制
原子性	Atomicity	事务中的操作要么全做要么全不做	undo log
一致性	Consistency	事务前后数据库从一个一致状态到另一个一致状态	其他三个特性共同保证
隔离性	Isolation	并发事务之间互不干扰	锁 + MVCC
持久性	Durability	事务提交后数据永久保存	redo log

易错点：

- 不清楚一致性是靠其他三个特性共同保证的（不是单独某个机制实现）
- 混淆原子性和一致性
- 认为事务一定能保证数据正确（逻辑错误不在ACID范畴内）
- 不了解 undo log 和 redo log 的分工

题目32：MySQL 的四种隔离级别是什么？

题干：列出MySQL支持的四种事务隔离级别，说明每种级别能解决和不能解决的并发问题。

参考答案：



复制代码

```
-- 查看当前隔离级别
SELECT @@transaction_isolation;

-- 设置隔离级别
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

隔离级别	脏读	不可重复读	幻读	说明
READ UNCOMMITTED	可能	可能	可能	最低级别，几乎不用
READ COMMITTED	不会	可能	可能	Oracle默认级别
REPEATABLE READ	不会	不会	可能*	MySQL默认级别
SERIALIZABLE	不会	不会	不会	串行化，性能最差

- 注：MySQL InnoDB 在 RR 级别下通过 MVCC + Next-Key Lock 在很大程度上避免了幻读。

逻辑解析：

- **脏读**：读到其他事务未提交的数据
- **不可重复读**：同一事务中两次读取同一行，结果不同（被其他事务UPDATE）
- **幻读**：同一事务中两次范围查询，结果集行数不同（被其他事务INSERT）
- MySQL默认 REPEATABLE READ，通过 MVCC 实现快照读，通过 Next-Key Lock 实现当前读的幻读防护

易错点：

- 不知道MySQL默认是RR而非RC
- 混淆不可重复读和幻读（前者是UPDATE导致，后者是INSERT/DELETE导致）
- 认为RR级别完全不会幻读（MVCC快照读不会，但当前读仍可能）
- 不了解各级别的性能开销递增关系

题目33：什么是MVCC？它是如何工作的？

题干：解释 InnoDB 的 MVCC（多版本并发控制）机制的实现原理。

参考答案：



复制代码

```
-- MVCC相关的隐藏字段（每行数据都有）：
-- DB_TRX_ID：最近修改该行的事务ID
-- DB_ROLL_PTR：指向undo log中该行的上一个版本
```

```
-- DB_ROW_ID: 隐含的自增ID（无主键时使用）

-- 可见性判断（Read View）：
-- m_ids: 生成Read View时活跃的事务ID列表
-- min_trx_id: m_ids中最小的事务ID
-- max_trx_id: 生成Read View时系统分配给下一个事务的ID
-- creator_trx_id: 创建该Read View的事务ID
```

逻辑解析：

MVCC核心组件：

1. 隐藏字段：每行都有 `trx_id` 和 `roll_ptr`
2. **Undo Log**版本链：通过 `roll_ptr` 形成链表，存储历史版本
3. **Read View**（读视图）：决定当前事务能看到哪些版本

可见性规则：

- `trx_id == creator_trx_id` -> 可见（自己修改的）
- `trx_id < min_trx_id` -> 可见（事务已提交）
- `trx_id >= max_trx_id` -> 不可见（事务在Read View后开始）
- `min_trx_id <= trx_id < max_trx_id` -> 检查是否在 `m_ids` 中（在则不可见，不在则可见）

RC vs RR 的区别：

- RC：每次 `SELECT` 都生成新的 Read View
- RR：只在事务的第一次 `SELECT` 时生成 Read View，后续复用

易错点：

- 不理解 Read View 的生成时机（RC每次读，RR第一次读）
- 混淆"快照读"（普通`SELECT`，用MVCC）和"当前读"（`SELECT FOR UPDATE`，用锁）
- 以为 MVCC 能解决所有并发问题（实际写写冲突仍需锁）

题目34：MySQL中有哪些锁？

题干：列举MySQL InnoDB中的锁类型并说明其用途。

参考答案：

```
-- 查看当前锁状态
SELECT * FROM performance_schema.data_locks;

-- 显式加锁示例
```

 复制代码

```
SELECT * FROM users WHERE id = 1 FOR UPDATE;      -- 排他锁（X锁）
SELECT * FROM users WHERE id = 1 LOCK IN SHARE MODE; -- 共享锁（S锁）
```

逻辑解析：

按粒度分类：

锁类型	粒度	说明
全局锁	数据库	FLUSH TABLES WITH READ LOCK, 全库只读
表级锁	表	元数据锁(MDL)、意向锁(IS/IX)、AUTO-INC锁
行级锁	行	记录锁、间隙锁、Next-Key Lock

按兼容性分类：

	S锁	X锁
S锁	兼容	冲突
X锁	冲突	冲突

InnoDB行级锁详解：

- **Record Lock（记录锁）**：锁住索引中的一行记录
- **Gap Lock（间隙锁）**：锁住索引记录之间的间隙，防止插入
- **Next-Key Lock**：Record Lock + Gap Lock，锁记录本身及其前面的间隙

易错点：

- 认为InnoDB都是行锁（没有索引时退化为表锁）
- 不清楚意向锁的作用（辅助表锁判断，避免逐行检查）
- 混淆 Gap Lock 和 Next-Key Lock
- 不了解锁是加在索引上而非数据行上

题目35：什么是死锁？如何避免和解决？

题干：解释死锁的概念，举例说明死锁的发生条件，并给出预防和处理方案。

参考答案：

复制代码

```
-- 死锁示例：
-- 事务A:
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1; -- 锁住id=1
UPDATE accounts SET balance = balance + 100 WHERE id = 2; -- 等待id=2的
锁

-- 事务B（同时执行）：
START TRANSACTION;
UPDATE accounts SET balance = balance - 50 WHERE id = 2; -- 锁住id=2
UPDATE accounts SET balance = balance + 50 WHERE id = 1; -- 等待id=1的
锁

-- 互相等待 = 死锁！

-- 查看死锁信息
SHOW ENGINE INNODB STATUS;

-- 查看锁等待超时时间
SHOW VARIABLES LIKE 'innodb_lock_wait_timeout';
```

逻辑解析：

死锁四个必要条件：

1. 互斥：资源被一个事务持有时不能被其他事务获取
2. 持有并等待：事务持有部分锁的同时等待其他锁
3. 不可抢占：已持有的锁不能被强制释放
4. 循环等待：事务间形成环形等待链

预防和处理方案：

- **固定加锁顺序**：所有事务按相同顺序访问表/行
- **缩短事务**：减少事务持有锁的时间
- **降低隔离级别**：RC 比 RR 产生的锁更少
- **InnoDB自动检测**：检测到死锁后回滚代价最小的事务（undo量最少的）
- **设置超时**：innodb_lock_wait_timeout（默认50秒）

易错点：

- 不清楚InnoDB会自动检测并解决死锁（回滚其中一个事务）
- 认为死锁一定是程序bug（高并发下正常可能发生，需要程序重试）

- 混淆死锁和锁等待超时
- 不知道如何通过 SHOW ENGINE INNODB STATUS 排查死锁原因

题目36：当前读和快照读的区别？

题干：解释MySQL中当前读和快照读的区别及各自的使用场景。

参考答案：

⌵

复制代码

```
-- 快照读 (Consistent Read)
SELECT * FROM users WHERE id = 1; -- 普通SELECT

-- 当前读 (Current Read / Locking Read)
SELECT * FROM users WHERE id = 1 FOR UPDATE;           -- 加X锁
SELECT * FROM users WHERE id = 1 LOCK IN SHARE MODE;    -- 加S锁 (MySQL 8.0: FOR SHARE)
INSERT INTO users VALUES (...);                        -- 隐式加X锁
UPDATE users SET name = 'Bob' WHERE id = 1;             -- 隐式加X锁
DELETE FROM users WHERE id = 1;                         -- 隐式加X锁
```

逻辑解析：

特性	快照读	当前读
读取的数据	Read View 对应的历史版本	最新已提交版本
是否加锁	不加锁	加共享/排他锁
并发性	高（读不阻塞写）	低（可能阻塞）
适用操作	普通 SELECT	SELECT FOR UPDATE/SHARE, INSERT/UPDATE/DELETE
实现机制	MVCC	锁

易错点：

- 认为所有 SELECT 都是快照读（FOR UPDATE/LOCK IN SHARE MODE 是当前读）
- 不清楚 UPDATE/DELETE 中的读取也是当前读
- 在 RR 级别下快照读和当前读可能看到不同数据（快照读看历史版本）

- 不理解"快照读不加锁"是 MVCC 实现高并发的关键

题目37: redo log 和 undo log 的作用?

题干: 解释 InnoDB 中 redo log 和 undo log 各自的作用和区别。

参考答案:

特性	redo log	undo log
作用	保证持久性 (Durability)	保证原子性 (Atomicity)
内容	物理日志 (数据页的修改)	逻辑日志 (逆操作)
写入时机	事务执行过程中写入	事务执行前写入
使用场景	崩溃恢复 (redo)	事务回滚 + MVCC
文件	ib_logfile0/1 (循环写入)	ibdata 或独立表空间

逻辑解析:

redo log (重做日志) :

- 记录"做了什么修改" (物理)
- 先写日志再写磁盘 (WAL - Write Ahead Log)
- 崩溃后重启时, 将已提交但未落盘的数据恢复
- 保证事务的持久性

undo log (回滚日志) :

- 记录"如何撤销" (逻辑)
- INSERT 对应 DELETE, UPDATE 对应反向 UPDATE
- 事务回滚时使用 undo log 撤销修改
- MVCC 通过 undo log 版本链实现快照读

易错点:

- 混淆 redo log 和 binlog (redo log 是InnoDB引擎层, binlog 是Server层)
- 不清楚 WAL 机制的意义 (将随机IO变为顺序IO, 提升性能)
- 以为 undo log 用完就删除 (MVCC需要保留到没有事务引用该版本)

题目38: binlog 和 redo log 的区别?

题干：对比 MySQL 的 binlog 和 InnoDB 的 redo log。

参考答案：

特性	binlog	redo log
层次	Server层（所有引擎共用）	InnoDB引擎层
内容	逻辑日志（SQL或行变更）	物理日志（数据页修改）
写入方式	追加写入（不覆盖）	循环写入（固定大小）
用途	主从复制、数据恢复	崩溃恢复
格式	STATEMENT/ROW/MIXED	固定格式



复制代码

```
-- 查看binlog格式
SHOW VARIABLES LIKE 'binlog_format';

-- 查看binlog列表
SHOW BINARY LOGS;

-- 查看binlog内容
SHOW BINLOG EVENTS IN 'mysql-bin.000001';
```

逻辑解析：

两阶段提交（保证 binlog 和 redo log 一致性）：

1. **Prepare阶段**：InnoDB 将 redo log 标记为 prepare 状态
2. **Commit阶段**：写 binlog -> 将 redo log 标记为 commit 状态

为什么需要两阶段提交：

- 如果先写 redo log 再写 binlog：崩溃后 redo log 恢复了但 binlog 没有，从库缺少该操作
- 如果先写 binlog 再写 redo log：崩溃后 binlog 有但数据没改，从库多了该操作

易错点：

- 不清楚为什么需要两种日志（历史原因 + 分层架构）
- 混淆两者的使用场景（redo log用于崩溃恢复，binlog用于复制和备份恢复）
- 不了解两阶段提交的必要性
- 认为有了 binlog 就不需要 redo log（各自解决不同问题）

题目39：如何解决幻读问题？

题干：在 MySQL InnoDB 的 REPEATABLE READ 级别下，如何解决幻读问题？

参考答案：

复制代码

```
-- 场景：事务A查询id>5的记录，事务B插入id=10的记录

-- 快照读：MVCC自动解决（Read View固定）
-- 事务A
START TRANSACTION;
SELECT * FROM users WHERE id > 5; -- 结果集A（快照读）
-- 事务B 插入 id=10 并提交
SELECT * FROM users WHERE id > 5; -- 仍然是结果集A（MVCC快照）
COMMIT;

-- 当前读：Next-Key Lock解决
-- 事务A
START TRANSACTION;
SELECT * FROM users WHERE id > 5 FOR UPDATE; -- 对(5, +∞)加Next-Key Lock
-- 事务B尝试 INSERT id=10 -> 被阻塞！
COMMIT;
-- 事务B插入成功
```

逻辑解析：

InnoDB在RR级别防止幻读的两种方式：

1. 快照读：通过MVCC的Read View，事务开始后看到的数据版本固定
2. 当前读：通过Next-Key Lock锁住记录及间隙，阻止其他事务在范围内插入

Next-Key Lock = Record Lock + Gap Lock：

- 假设索引中有值 1, 5, 10
- SELECT * WHERE id > 5 FOR UPDATE 会锁住：(5, 10]、(10, +∞)
- 其他事务无法在这些间隙中插入新记录

易错点：

- 认为RR级别完全解决了幻读（快照读解决了，但某些组合场景仍可能出现异常）
- 不清楚快照读和当前读分别用什么机制防幻读
- 混淆Gap Lock的锁定范围

- 不了解当前读后再快照读的场景下可能出现的数据不一致

题目40：乐观锁和悲观锁的区别及应用场景？

题干：对比乐观锁和悲观锁的实现方式及适用场景。

参考答案：

▼

复制代码

```
-- 悲观锁实现：SELECT FOR UPDATE
START TRANSACTION;
SELECT stock FROM products WHERE id = 1 FOR UPDATE; -- 加排他锁
-- 判断stock是否足够
UPDATE products SET stock = stock - 1 WHERE id = 1;
COMMIT;

-- 乐观锁实现：版本号机制
-- 方法1：版本号
UPDATE products
SET stock = stock - 1, version = version + 1
WHERE id = 1 AND version = #{oldVersion};
-- 如果 affected_rows = 0，说明被其他事务修改过，需要重试

-- 方法2：CAS（Compare And Swap）
UPDATE products
SET stock = stock - 1
WHERE id = 1 AND stock = #{oldStock};
```

逻辑解析：

特性	悲观锁	乐观锁
实现方式	数据库锁（FOR UPDATE）	版本号/CAS
加锁时机	读取数据时就加锁	更新时才检查
并发性能	低（阻塞等待）	高（不阻塞，但可能重试）
适用场景	写多读少、冲突频繁	读多写少、冲突较少
实现位置	数据库层	应用层

易错点：

- 乐观锁不是数据库提供的锁，而是应用层的并发控制策略
- 乐观锁重试次数需要限制，否则高冲突场景下性能反而差
- 悲观锁忘记开启事务（FOR UPDATE 必须在事务中才有效）
- CAS 的ABA问题（值改了又改回来，检测不到变化 -> 应用版本号更安全）

模块五：数据库设计与进阶（第41-50题）

题目41：三大范式是什么？

题干：解释数据库设计中的三大范式，并举例说明。

参考答案：

✓ 复制代码

```
-- 违反1NF（字段不可再分）
CREATE TABLE bad_1nf (
  id INT,
  address VARCHAR(200) -- '北京市海淀区xxx路' 无法单独查询城市
);

-- 符合1NF
CREATE TABLE good_1nf (
  id INT,
  province VARCHAR(50),
  city VARCHAR(50),
  street VARCHAR(100)
);

-- 违反2NF（非主属性完全依赖主键）
-- 订单明细表，主键为(order_id, product_id)
CREATE TABLE bad_2nf (
  order_id INT,
  product_id INT,
  product_name VARCHAR(100), -- 只依赖product_id，不依赖order_id（部分依赖）
  quantity INT,
  PRIMARY KEY(order_id, product_id)
);

-- 违反3NF（不存在传递依赖）
CREATE TABLE bad_3nf (
  student_id INT PRIMARY KEY,
  dept_id INT,
```

```
dept_name VARCHAR(50)  -- dept_name依赖dept_id, dept_id依赖student_id
(传递依赖)
);
```

逻辑解析：

范式	要求	解决的问题
1NF	字段不可再分（原子性）	数据冗余、查询困难
2NF	在1NF基础上，消除部分依赖	复合主键表中的冗余
3NF	在2NF基础上，消除传递依赖	非主键字段间的冗余

易错点：

- 实际开发中不一定严格遵循范式（适度冗余可提高查询性能）
- 混淆2NF和3NF的区别（2NF针对复合主键的部分依赖，3NF针对传递依赖）
- 认为满足3NF就是最优设计（有时需要反范式化）

题目42： VARCHAR 和 CHAR 的区别？

题干：对比 VARCHAR 和 CHAR 数据类型的区别，说明各自的适用场景。

参考答案：

▼

复制代码

```
-- CHAR：定长，不足时补空格
CREATE TABLE t1 (name CHAR(10));      -- 始终占用10字节
INSERT INTO t1 VALUES ('abc');        -- 存储为 'abc      '(补7个空格)

-- VARCHAR：变长，额外1-2字节存长度
CREATE TABLE t2 (name VARCHAR(10));  -- 'abc' 实际占4字节(3+1)
INSERT INTO t2 VALUES ('abc');        -- 存储为 'abc'
```

逻辑解析：

特性	CHAR(N)	VARCHAR(N)
----	---------	------------

存储方式	定长，不足补空格	变长，额外存储长度
空间	固定N字节	实际长度+1/2字节
检索效率	略高（偏移量固定）	略低
尾部空格	检索时去除	保留
适用场景	固定长度（MD5、身份证号）	长度不确定的字符串

N 的含义：

- MySQL 5.0+ 中 CHAR(N) 和 VARCHAR(N) 的 N 是字符数而非字节数
- 实际字节数取决于字符集（UTF-8: 1-3字节/字符，UTF8MB4: 1-4字节/字符）

易错点：

- 认为 VARCHAR(255) 和 VARCHAR(1000) 占用空间相同（索引、排序时会按最大长度分配内存）
- 不清楚 N 代表字符数而非字节数
- 忽视 VARCHAR 的长度前缀开销（ ≤ 255 字节用1字节， > 255 字节用2字节）
- CHAR 适合定长场景这一点经常被忽视

题目43：INT(11) 中的 11 代表什么？

题干：INT(11) 中的数字 11 是什么含义？它是否影响存储范围？

参考答案：



复制代码

```
-- INT(11) vs INT(4) 存储空间和范围完全相同
CREATE TABLE t1 (
  a INT(11),          -- 4字节，范围 -2^31 ~ 2^31-1
  b INT(4),           -- 4字节，范围 -2^31 ~ 2^31-1
  c INT(11) ZEROFILL -- 配合 ZEROFILL 时才有视觉效果
);

INSERT INTO t1 VALUES (42, 42, 42);
-- a: 42
-- b: 42
-- c: 00000000042 (ZEROFILL补零到11位)
```

逻辑解析：

- INT(11) 中的 11 是**显示宽度**，不影响存储空间和取值范围
- 只在配合 `ZEROFILL` 属性时才有效果（不足位数时补零显示）
- INT 始终占 4 字节，范围固定为 -2147483648 到 2147483647
- MySQL 8.0.17+ 已废弃整数类型的显示宽度定义

易错点：

- 认为 INT(4) 只能存4位数（错误！范围与 INT(11) 完全一样）
- 混淆 INT(N) 和 VARCHAR(N) 中 N 的含义
- 不知道该特性已被废弃（MySQL 8.0.17+）

题目44：什么是分库分表？什么时候需要？

题干：解释分库分表的概念、策略及适用场景。

参考答案：

⌵

复制代码

```
-- 水平分表示例（按用户ID取模）
-- user_0, user_1, user_2, user_3
-- 路由规则: table_index = user_id % 4

-- 分表前
SELECT * FROM users WHERE id = 12345;

-- 分表后（路由到 user_1, 因为 12345 % 4 = 1）
SELECT * FROM users_1 WHERE id = 12345;

-- 垂直分表示例（按字段冷热分离）
-- users_basic: id, name, phone, status（高频访问）
-- users_detail: id, bio, avatar_url, address（低频访问）
```

逻辑解析：

策略	说明	适用场景
垂直分表	按字段拆分到不同表	字段多、冷热数据明显
水平分表	按行拆分到多个结构相同的表	单表数据量过大（>2000万行）

垂直分库	按业务拆分到不同数据库	微服务架构
水平分库	按规则将数据分散到多个库	写入压力大、连接数不足

常用分片策略：

- **取模**：均匀分布，但扩容不方便
- **范围**：扩容方便，但可能热点不均
- **一致性Hash**：扩容只迁移少量数据

易错点：

- 过早分库分表（单表千万级以下通常不需要）
- 分表后跨表 JOIN、分布式事务、全局排序变得复杂
- 忽视分库分表带来的运维复杂度
- 不考虑全局唯一ID生成方案（雪花算法等）

题目45：MySQL主从复制的原理？

题干：解释MySQL主从复制的工作流程和实现原理。

参考答案：



复制代码

```
-- 主库配置 (my.cnf)
-- server-id=1
-- log-bin=mysql-bin
-- binlog-format=ROW

-- 从库配置
CHANGE MASTER TO
  MASTER_HOST='master_ip',
  MASTER_USER='repl_user',
  MASTER_PASSWORD='password',
  MASTER_LOG_FILE='mysql-bin.000001',
  MASTER_LOG_POS=154;

START SLAVE;
SHOW SLAVE STATUS\G
```

逻辑解析：

主从复制三线程模型：

1. **Binlog Dump Thread (主库)**：将 binlog 事件发送给从库
2. **IO Thread (从库)**：接收主库的 binlog 并写入本地 relay log
3. **SQL Thread (从库)**：读取 relay log 并重放 SQL 到从库数据

复制流程：



复制代码

主库写入 -> binlog -> Dump线程推送 -> 从库IO线程 -> relay log -> SQL线程重放 -> 从库数据

复制模式：

- **异步复制**：主库不等从库确认（默认，可能丢数据）
- **半同步复制**：至少一个从库确认收到 relay log
- **组复制**：多数节点确认（MGR）

易错点：

- 主从延迟是常见问题（从库压力大、大事务、网络延迟）
- 读写分离时可能读到旧数据（从库延迟导致）
- 不清楚 relay log 的作用（缓冲作用，IO线程和SQL线程解耦）
- 混淆 binlog 和 relay log

题目46：什么是慢查询日志？如何使用？

题干：解释MySQL慢查询日志的配置和使用方法。

参考答案：



复制代码

```
-- 查看慢查询相关配置
SHOW VARIABLES LIKE 'slow_query%';
SHOW VARIABLES LIKE 'long_query_time';

-- 动态开启慢查询日志
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 1; -- 超过1秒的查询记录
SET GLOBAL slow_query_log_file = '/var/log/mysql/slow.log';

-- 记录没有使用索引的查询
SET GLOBAL log_queries_not_using_indexes = 'ON';
```

[复制代码](#)

使用 `mysqldumpslow` 分析慢查询日志

`mysqldumpslow -s t -t 10 /var/log/mysql/slow.log` # 按时间排序取前10

`mysqldumpslow -s c -t 10 /var/log/mysql/slow.log` # 按次数排序取前10

逻辑解析：

慢查询日志使用流程：

1. 开启慢查询日志：配置阈值和日志文件路径
2. 收集一段时间：让系统正常运行积累慢查询
3. 分析日志：使用 `mysqldumpslow` 或 `pt-query-digest` 分析
4. 优化：EXPLAIN 分析 -> 加索引/改SQL -> 验证效果

易错点：

- 生产环境不建议 `long_query_time` 设太小（日志量大，影响IO）
- 慢查询日志会有一定性能开销，问题解决后建议关闭
- 不知道有 `pt-query-digest` 这样更强大的分析工具
- 只看执行时间，忽视执行次数（高频慢查询优先优化）

题目47：大表如何添加索引而不锁表？

题干：线上大表（千万级数据）需要添加索引，如何避免长时间锁表影响业务？

参考答案：

[复制代码](#)

-- MySQL 5.6+ 支持 Online DDL (ALGORITHM=INPLACE)

`ALTER TABLE big_table ADD INDEX idx_name(name), ALGORITHM=INPLACE, LOCK=NONE;`

-- 如果Online DDL不支持，使用`pt-online-schema-change`

-- `pt-online-schema-change --alter "ADD INDEX idx_name(name)" \`
-- `D=mydb,t=big_table --execute`

-- MySQL 8.0+ 也可以使用 `gh-ost` (GitHub开源工具)

逻辑解析：

方案	原理	是否锁表	适用版本
Online DDL	InnoDB原生支持, INPLACE算法	短暂元数据锁	5.6+
pt-online-schema-change	创建新表->触发器同步->切换	不锁表	所有版本
gh-ost	binlog回放代替触发器	不锁表	所有版本

Online DDL 流程：

1. 获取 MDL 写锁（瞬间）
2. 降级为 MDL 读锁，开始DDL操作
3. 期间DML操作记录到 online log
4. DDL完成后重放 online log
5. 升级为 MDL 写锁完成切换（瞬间）

易错点：

- 不是所有 ALTER TABLE 都支持 ALGORITHM=INPLACE（如改字段类型通常需要 COPY）
- Online DDL 虽然不完全锁表，但仍有短暂的 MDL 锁
- pt-osc 在高并发时触发器可能造成压力
- 不检查磁盘空间就执行（新表 + 日志可能需要双倍空间）

题目48：如何设计一个高效的分页查询？

题干： `LIMIT 1000000, 10` 为什么慢？如何优化深度分页？

参考答案：



复制代码

```
-- 低效：深度分页
SELECT * FROM orders ORDER BY id LIMIT 1000000, 10;
-- 需要扫描1000010行，丢弃前1000000行

-- 优化方案1：游标分页（推荐）
-- 前端传递上一页最后一条的id
SELECT * FROM orders WHERE id > 1000000 ORDER BY id LIMIT 10;
-- 利用主键索引直接定位，效率极高
```

```
-- 优化方案2：延迟关联（子查询覆盖索引）
SELECT o.* FROM orders o
INNER JOIN (
    SELECT id FROM orders ORDER BY id LIMIT 1000000, 10
) AS tmp ON o.id = tmp.id;
-- 子查询只扫描索引（覆盖索引），减少回表

-- 优化方案3：业务限制
-- 限制用户只能翻前100页（如Google搜索结果）
```

逻辑解析：

LIMIT offset, size 的执行过程：

1. 从磁盘/索引读取 offset + size 行
2. 丢弃前 offset 行
3. 返回剩余 size 行

offset 越大，浪费越多。

优化方案	适用场景	限制
游标分页	有连续递增ID	不能跳页
延迟关联	通用	性能提升有限（仍需扫描）
业务限制	产品可接受	需要产品配合
搜索引擎	大数据量搜索	需要额外中间件

易错点：

- 游标分页要求排序字段有索引且唯一（否则可能漏数据或重复）
- 延迟关联的子查询必须能走覆盖索引才有效
- 深度分页问题不只是 LIMIT 的问题，排序也可能很耗时
- 不考虑数据变更导致分页不一致的问题

题目49：MySQL 如何存储 NULL 值？NULL 相关的注意事项？

题干：解释MySQL中NULL值的存储方式，以及使用NULL时的常见陷阱。

参考答案：

```
-- NULL 的比较
SELECT NULL = NULL;      -- 结果: NULL (不是TRUE)
SELECT NULL != NULL;    -- 结果: NULL (不是TRUE)
SELECT NULL IS NULL;    -- 结果: 1 (TRUE)
SELECT NULL IS NOT NULL;-- 结果: 0 (FALSE)

-- NULL 与聚合函数
SELECT COUNT(NULL);      -- 0
SELECT SUM(NULL);        -- NULL
SELECT AVG(NULL);        -- NULL
SELECT COUNT(*);         -- 统计所有行 (包含NULL)
SELECT COUNT(col);       -- 只统计col非NULL的行

-- NULL 与运算
SELECT 1 + NULL;         -- NULL
SELECT CONCAT('a', NULL);-- NULL (MySQL中)
SELECT IFNULL(NULL, 0);  -- 0 (IFNULL替换NULL值)
SELECT COALESCE(NULL, NULL, 'default'); -- 'default'
```

逻辑解析:

NULL 存储方式 (InnoDB) :

- 每行数据有一个 NULL 标志位图 (bitmap) , 每个可为 NULL 的列占1个bit
- NULL 列不实际存储数据, 只在bitmap中标记
- 因此 NULL 比空字符串 '' 或 0 更省空间

常见陷阱	说明
NULL = NULL 为 NULL	不能用 = 比较NULL, 必须用 IS NULL
NOT IN 含NULL	NOT IN (1, NULL) 永远返回空结果集
COUNT(col)	忽略NULL行
索引对NULL	B+树可以索引NULL值 (常见误解认为不可以)
UNIQUE约束	允许多个NULL (NULL != NULL)

易错点:

- 用 `= NULL` 代替 `IS NULL` 判断空值
- WHERE 条件中 NOT IN 子查询含 NULL 导致查不到任何结果

- 认为 NULL 列不能建索引（可以的）
- UNIQUE 索引允许多个 NULL 值（因为 NULL != NULL）
- CONCAT 遇到 NULL 结果整个变 NULL（需用 CONCAT_WS 或 IFNULL 处理）

题目50：如何设计一个评论系统的数据库表？

题干：设计一个支持多级嵌套评论的数据库表结构，要求支持高效的查询和分页。

参考答案：

复制代码

-- 方案1：邻接表模型（简单直观）

```
CREATE TABLE comments (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  content TEXT NOT NULL,  
  user_id BIGINT NOT NULL,  
  article_id BIGINT NOT NULL,  
  parent_id BIGINT DEFAULT NULL COMMENT '父评论ID, NULL为顶级评论',  
  root_id BIGINT DEFAULT NULL COMMENT '根评论ID, 方便查询整棵树',  
  depth TINYINT DEFAULT 0 COMMENT '嵌套层级',  
  like_count INT DEFAULT 0,  
  status TINYINT DEFAULT 1 COMMENT '1正常 0删除',  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_article_root(article_id, root_id, created_at),  
  INDEX idx_parent(parent_id),  
  INDEX idx_user(user_id)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- 查询某篇文章的顶级评论（分页）

```
SELECT * FROM comments  
WHERE article_id = 1001 AND parent_id IS NULL AND status = 1  
ORDER BY created_at DESC  
LIMIT 20;
```

-- 查询某条评论的所有子评论（按根评论聚合）

```
SELECT * FROM comments  
WHERE root_id = 5001 AND status = 1  
ORDER BY created_at ASC;
```

-- 方案2：路径枚举（适合展示完整树）

```
CREATE TABLE comments_path (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  content TEXT NOT NULL,
```

```

user_id BIGINT NOT NULL,
article_id BIGINT NOT NULL,
path VARCHAR(500) COMMENT '路径如 /1/5/12/ 表示层级关系',
depth TINYINT DEFAULT 0,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
INDEX idx_article_path(article_id, path)
);

-- 查询某条评论的所有子孙评论
SELECT * FROM comments_path
WHERE article_id = 1001 AND path LIKE '/1/5/%'
ORDER BY path, created_at;

```

逻辑解析：

方案	优点	缺点	适用场景
邻接表+root_id	简单，增删快	查完整子树需递归或多次查询	两级评论（类微博）
路径枚举	查子树方便	路径长度受限，修改父节点代价大	展示完整树（类Reddit）
闭包表	查询灵活	空间开销大	需要频繁查祖先/后代

实际工程选择：

- 大多数场景用邻接表 + root_id即可（类微博/微信的两级评论）
- 加 root_id 字段将一棵评论树归属到根评论，查询时先查顶级评论再按 root_id 加载子树
- 配合 Redis 缓存热门评论，减少数据库压力

易错点：

- 无限递归嵌套性能极差（应限制最大层级）
- 忘记加 article_id 的索引导致全表扫描
- 删除评论时是物理删除还是软删除需要提前设计
- 排序需求（按时间/按热度）需要不同的索引策略
- 不考虑评论数据量增长（大V文章可能有上万评论）

复习建议

复习阶段	模块	建议
第一阶段	模块一二（SQL编写题）	在LeetCode Database题目中实际练习，做到手写无误
第二阶段	模块三四（原理概念题）	理解底层实现，用自己的话能讲清楚流程和原因
第三阶段	模块五（设计题）	结合项目经验，能说出实际应用场景和方案权衡